1. Modelagem com Embedding

Enunciado: Modele um documento que represente um time de eSports com seus jogadores incorporados. O time deve ter nome, modalidade e um array de jogadores contendo nickname, idade e função (função in-game, como suporte, atirador etc.).

```
use MegaArena

db.times.insertOne({
    nome: "Meisters",
    modalidade: "CS2",
    jogadores: [
        { nickname: "Enkidu", idade: 21, funcao: "Entry Fragger" },
        { nickname: "GyaruAisuru", idade: 23, funcao: "Lurker" },
        { nickname: "Mihael", idade: 20, funcao: "AWPer" },
        { nickname: "Ankara!", idade: 17, funcao: "Suporte"},
        { nickname: "Elminster", idade: 22, funcao: "IGL"}
    ]
})

db.jogadores.insertMany([
    { _id: 1, nickname: "Enkidu", idade: 21, funcao: "Entry Fragger" },
    { _id: 2, nickname: "GyaruAisuru", idade: 23, funcao: "Lurker" },
    { _id: 3, nickname: "Mihael", idade: 20, funcao: "AWPer" },
    { _id: 4, nickname: "Ankara!", idade: 17, funcao: "Suporte"},
    { _id: 5, nickname: "Elminster", idade: 22, funcao: "IGL"}
])
```

2. Modelagem com Referencing

Enunciado: Modele um relacionamento 1:N entre coleções jogadores e estatisticas usando referência, onde cada documento de estatísticas representa um jogo com informações como eliminações, mortes e assistências

```
db.estatisticas.insertMany([
    { IDJ: 1, eliminacoes: 22, mortes: 15, assistencias: 3 },
    { IDJ: 2, eliminacoes: 12, mortes: 14, assistencias: 10 },
    { IDJ: 3, eliminacoes: 28, mortes: 10, assistencias: 2 },
    { IDJ: 5, eliminacoes: 30, mortes: 12, assistencias: 1 }
])
```

3. Consulta com Operadores Lógicos

Enunciado: Recupere todos os jogadores que atuam como "suporte" OU "atirador" E têm mais de 18 anos.

Como CS não tem a função atirador em específico, acabei trocando

```
db.jogadores.find({
$and: [
    {$or: [
        { funcao: "Suporte" },
        { funcao: "AWPer" }
```

```
    ]},
    {idade: { $gt: 18 }}
]
})
```

4. Atualização com Operadores

Enunciado: Adicione o campo status: "ativo" a todos os times que tenham pelo menos 5 jogadores.

```
db.times.updateMany(
    { jogadores: { $size: 5 } },
    { $set: { status: "ativo" } }
)
```

5. Exclusão com Condição

Enunciado: Remova todos os jogadores que ainda não participaram de nenhuma partida (sem estatísticas registradas).

```
var jogadoresAtivos = db.estatisticas.distinct("IDJ");

db.jogadores.deleteMany({
    _id: { $nin: jogadoresAtivos }
});
```

6. Agregação e Projeção

Enunciado: Agrupe as estatísticas por jogador e calcule a média de eliminações. Projete somente o nickname e a média calculada.

```
db.estatisticas.aggregate([
  {
    $group: {
      _id: "$IDJ",
      mediaEliminacoes: { $avg: "$eliminacoes" }
    }
  },
  {
    $lookup: {
      from: "jogadores",
      localField: "_id",
      foreignField: "_id",
      as: "detalhes_jogador"
    }
  },
  { $unwind: "$detalhes_jogador" },
  {
    $project: {
      _id: 0,
      nickname: "$detalhes_jogador.nickname",
      mediaEliminacoes: 1
```

```
        }
    }
])
```

7. Indexação e Otimização

Enunciado: Crie um índice no campo nickname da coleção jogadores e explique, com consulta, como isso melhora o desempenho em filtros.

```
db.jogadores.createIndex({ nickname: 1 })
```

```
db.jogadores.find({ nickname: "GyaruAisuru" }).explain("executionStats")
```

Usando índice, o banco vai direto ao ponto no funcionamento da consulta, pois sem o índice ele varre todo o banco até achar o que quer

8. Paginação

Enunciado: Liste os 5 primeiros jogadores de uma equipe ordenados por idade decrescente.

Ignore os 3 primeiros resultados.

```
db.times.aggregate([
    { $match: { nome: "Meisters" } },
    { $unwind: "$jogadores" },
```

```
    { $sort: { "jogadores.idade": -1 } },
    { $skip: 3 },
    { $limit: 5 },
    { $replaceRoot: { newRoot: "$jogadores" } }
])
```